Evaluating models

🎯 Today's goal

By the end of this hour you will:

- see why **accuracy can lie** when one class is rare
- read a **confusion matrix**: true/false positives and negatives
- calculate **precision**, **recall** and **F1**, by hand and with scikit-learn
- move the **decision threshold** to catch more of the cases that matter
- get a fairer score with **cross-validation**

🤔 Why this matters

Imagine a smoke alarm that *never* rings. In a normal year there is no fire on 364 days, so the alarm is "right" 99.7% of the time. It is also completely useless: on the one day that matters, it stays silent.

Models are the same. One number (accuracy) hides **which kind** of mistake the model makes. Missing a sick patient is much worse than sending a healthy person for one more check. Today you'll learn the tools data scientists use to see the full picture before they trust a model.

🎭 When accuracy lies

Here is a **made-up** screening of 200 people, where only 10 are sick (5%). In the label, `1` means "sick" and `0` means "healthy". A lazy "model" simply says *healthy* to everybody:

```
import numpy as np

# Made-up data: 190 healthy people (0) and 10 sick people (1)
y_screen = np.array([0] * 190 + [1] * 10)

lazy_pred = np.zeros(200, dtype=int)     # always predicts 0 = "healthy"

accuracy = (lazy_pred == y_screen).mean()
print(f"Lazy model accuracy: {accuracy:.0%}")
found = ((lazy_pred == 1) & (y_screen == 1)).sum()   # said sick AND really sick
print("Sick people it found:", found, "of", y_screen.sum())
```

```
Lazy model accuracy: 95%
Sick people it found: 0 of 10
```

95% accuracy, and not a single sick person found. When one class is much rarer than the other, we call the data **imbalanced** (fraud, rare diseases, machine failures…). On imbalanced data, always compare with this "say the common answer" baseline (Day 16), and look deeper than accuracy.

Reminder: `{accuracy:.0%}` shows a number as a percentage with 0 decimals (Day 15).

🧩 The four kinds of answer

For a yes/no question there are exactly four possibilities. We call the class we're looking for (sick) the **positive** class. "Positive" doesn't mean good; it means "the thing we're searching for".

	Model says healthy (0)	**Model says sick (1)**
Really healthy (0)	**TN**: true negative ✓	**FP**: false positive ✗ a false alarm
Really sick (1)	**FN**: false negative ✗ a missed case	**TP**: true positive ✓

This 2 × 2 table is called the **confusion matrix** (it shows where the model gets "confused"). Let's count the four boxes by hand for 12 made-up patients, using the boolean masks from Day 9 (`&` means "and"):

```
# Made-up results for 12 patients
actual    = np.array([1, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0])
predicted = np.array([1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 1, 0])

tp = ((actual == 1) & (predicted == 1)).sum()
fp = ((actual == 0) & (predicted == 1)).sum()
fn = ((actual == 1) & (predicted == 0)).sum()
tn = ((actual == 0) & (predicted == 0)).sum()

print(f"TP={tp}  FP={fp}  FN={fn}  TN={tn}")
print("Accuracy:", (tp + tn) / len(actual))
```

```
TP=3  FP=2  FN=1  TN=6
Accuracy: 0.75
```

scikit-learn builds the whole table with `confusion_matrix(actual, predicted)`. The **rows** are the real classes, the **columns** are the predictions, in the order 0 then 1. `.ravel()` flattens it into the four numbers, always in the order TN, FP, FN, TP:

```
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(actual, predicted)
print(cm)

tn, fp, fn, tp = cm.ravel()
print(f"TN={tn}  FP={fp}  FN={fn}  TP={tp}")
```

```
[[6 2]
 [1 3]]
TN=6  FP=2  FN=1  TP=3
```

🎯 Precision, recall and F1

From the four counts we build three famous scores. Each answers a different question:

Score	Formula	The question it answers
Precision	`TP / (TP + FP)`	When the model says "sick", how often is it right? (few false alarms)
Recall	`TP / (TP + FN)`	Of all the really sick people, how many did it find? (few missed cases)
F1	`2 × P × R / (P + R)`	One number that is only high when precision *and* recall are both high

```
precision = tp / (tp + fp)
recall = tp / (tp + fn)
f1 = 2 * precision * recall / (precision + recall)

print(f"Precision: {precision:.2f}   (of {tp + fp} alarms, {tp} were real)")
print(f"Recall:    {recall:.2f}   (of {tp + fn} sick people, {tp} were found)")
print(f"F1:        {f1:.2f}")
```

```
Precision: 0.60   (of 5 alarms, 3 were real)
Recall:    0.75   (of 4 sick people, 3 were found)
F1:        0.67
```

F1 is a special kind of average (the "harmonic mean") that punishes a weak side: a model with precision 1.0 and recall 0.1 gets an F1 of only 0.18, not 0.55. scikit-learn has a function for each score. Let's also score the lazy model from the start:

```
from sklearn.metrics import precision_score, recall_score, f1_score

print("12 patients, precision:", precision_score(actual, predicted))
print("12 patients, recall:   ", recall_score(actual, predicted))
print("12 patients, F1:       ", round(f1_score(actual, predicted), 2))

# zero_division=0: the lazy model never says "sick", so precision would be 0 / 0
print("Lazy model recall:", recall_score(y_screen, lazy_pred))
print("Lazy model F1:    ", f1_score(y_screen, lazy_pred, zero_division=0))
```

```
12 patients, precision: 0.6
12 patients, recall:    0.75
12 patients, F1:        0.67
Lazy model recall: 0.0
Lazy model F1:     0.0
```

The 95%-accurate lazy model has a recall and F1 of **0**. Now its uselessness is visible.

Which one matters? It depends on the cost of each mistake. Missing a sick patient or a fraud is expensive → care most about **recall**. A spam filter that hides an important email is annoying → care most about **precision**. No clear winner → look at **F1**.

🔬 Evaluating a real model

Back to the breast cancer teaching dataset from Day 18 (a teaching example, not medical advice). In this dataset `0` means malignant and `1` means benign. We want the positive class to be "the thing we're searching for", so we flip it: `1` = malignant from now on. Then the same scaled logistic regression pipeline as on Day 18:

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer(as_frame=True)
X = cancer.data
y = (cancer.target == 0).astype(int)      # 1 = malignant (positive), 0 = benign
print("Counts:", y.value_counts().to_dict())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print(f"Test accuracy: {model.score(X_test, y_test):.3f}")
```

```
Counts: {0: 357, 1: 212}
Test accuracy: 0.965
```

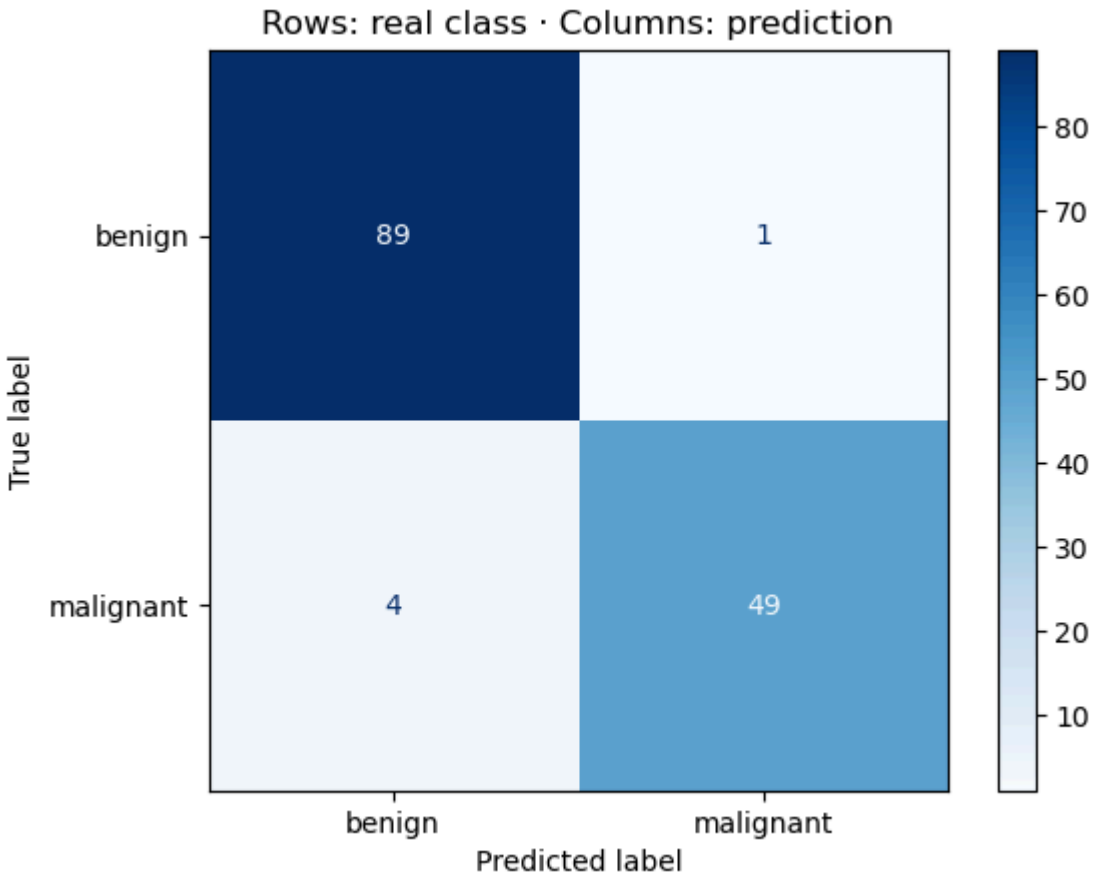
The accuracy looks great. But which mistakes are hiding inside it? A confusion matrix as a picture makes it easy to see:

```
import matplotlib.pyplot as plt
from sklearn.metrics import ConfusionMatrixDisplay

print(confusion_matrix(y_test, y_pred))

ConfusionMatrixDisplay.from_predictions(
    y_test, y_pred, display_labels=["benign", "malignant"], cmap="Blues")
plt.title("Rows: real class · Columns: prediction")
plt.show()
```

```
[[89  1]
 [ 4 49]]
```



The bottom-left box is the one to worry about: malignant cases the model called benign (false negatives). `classification_report` prints precision, recall and F1 for *each* class at once. `support` is how many test rows are really in that class:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=["benign", "malignant"]))
```

```
              precision    recall  f1-score   support

      benign       0.96      0.99      0.97        90
   malignant       0.98      0.92      0.95        53

    accuracy                           0.97       143
   macro avg       0.97      0.96      0.96       143
weighted avg       0.97      0.97      0.96       143
```

Every score function wants the real answers and the predictions for the **same rows**, in the same order. If the two lists have different lengths, you get an error worth recognising:

💡 This example has a mistake on purpose. Run it and read the error.

```
recall_score(y_test, y_pred[:100])      # oops: only the first 100 predictions
```

⚠️ **ValueError**

The kind of value is right but the value itself isn't, like int("hello").

```
Traceback (most recent call last):
  line 1, in <module>
    recall_score(y_test, y_pred[:100])      # oops: only the first 100
predictions
    ~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^
ValueError: Found input variables with inconsistent numbers of samples: [143,
100]
```

"Found input variables with inconsistent numbers of samples" means: the two lists don't have the same length.

🎚️ Choosing a threshold

On Day 18 you saw that `predict_proba` gives a probability, and `predict` says "1" when that probability is at least **0.5**. That 0.5 is the **threshold**, and nothing forces us to keep it. A lower threshold flags more people: recall goes up, but so do false alarms. Let's try several (`missed` = false negatives, `alarms` = false positives):

```
proba = model.predict_proba(X_test)[:, 1]     # probability of class 1 (malignant)

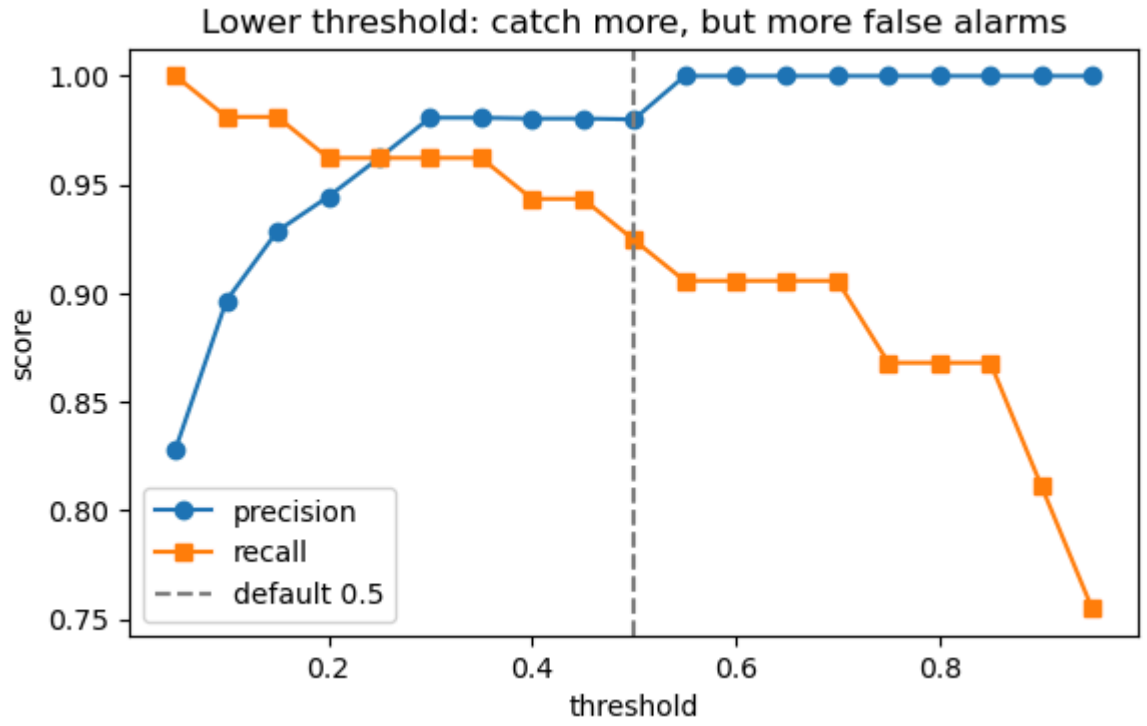
for threshold in [0.9, 0.7, 0.5, 0.3, 0.1]:
    pred_t = (proba >= threshold).astype(int)
    p = precision_score(y_test, pred_t)
    r = recall_score(y_test, pred_t)
    tn, fp, fn, tp = confusion_matrix(y_test, pred_t).ravel()
    print(f"t={threshold}: precision={p:.3f} recall={r:.3f} missed={fn} alarms={fp}")
```

```
t=0.9: precision=1.000 recall=0.811 missed=10 alarms=0
t=0.7: precision=1.000 recall=0.906 missed=5 alarms=0
t=0.5: precision=0.980 recall=0.925 missed=4 alarms=1
t=0.3: precision=0.981 recall=0.962 missed=2 alarms=1
t=0.1: precision=0.897 recall=0.981 missed=1 alarms=6
```

A chart shows the **trade-off**: as the threshold goes down, recall rises and precision falls.

```
thresholds = np.linspace(0.05, 0.95, 19)
precisions = []
recalls = []
for t in thresholds:
    pred_t = (proba >= t).astype(int)
    precisions.append(precision_score(y_test, pred_t))
    recalls.append(recall_score(y_test, pred_t))

plt.figure(figsize=(6.5, 3.8))
plt.plot(thresholds, precisions, marker="o", label="precision")
plt.plot(thresholds, recalls, marker="s", label="recall")
plt.axvline(0.5, color="gray", linestyle="--", label="default 0.5")
plt.xlabel("threshold")
plt.ylabel("score")
plt.title("Lower threshold: catch more, but more false alarms")
plt.legend()
plt.show()
```



Who chooses the threshold? Not the computer: people do, based on the cost of each mistake. For a first screening, a doctor may accept more false alarms to miss fewer cases.

🔁 Cross-validation: more than one exam

Our test score depends on *which* rows happened to land in the test set (you saw this with the jumpy trees on Day 19). **Cross-validation** gives a fairer score. With `cv=5`:

1. The data is cut into 5 equal parts, called **folds**.

2. The model is trained 5 times. Each time, one fold is the test set and the other 4 are for training.
3. You get 5 scores. Their **mean** is the fair estimate, and their spread (standard deviation, Day 9) shows how steady the model is.

```
from sklearn.model_selection import cross_val_score

cv_model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
scores = cross_val_score(cv_model, X, y, cv=5)

print("5 scores:", np.round(scores, 3))
print(f"Mean: {scores.mean():.3f}  (spread ± {scores.std():.3f})")

recall_scores = cross_val_score(cv_model, X, y, cv=5, scoring="recall")
print(f"Mean recall: {recall_scores.mean():.3f}")
```

```
5 scores: [0.982 0.982 0.974 0.974 0.991]
Mean: 0.981  (spread ± 0.007)
Mean recall: 0.962
```

Notice we give `cross_val_score` the *whole* `X` and `y`; it does the splitting itself. `scoring="recall"` (or `"precision"`, `"f1"`...) chooses the score.

📐 One more score: ROC AUC (briefly)

Precision and recall depend on the threshold. **ROC AUC** looks at *all* thresholds at once and asks: if I pick one random malignant case and one random benign case, how often does the model give the malignant one the higher probability? `1.0` is perfect, `0.5` is coin-flipping. It needs the probabilities, not the 0/1 predictions:

```
from sklearn.metrics import roc_auc_score

print(f"ROC AUC: {roc_auc_score(y_test, proba):.3f}")
```

```
ROC AUC: 0.996
```

📋 A model-evaluation checklist

1. How balanced are the classes? Compare with the "always the common answer" baseline.
2. Look at the confusion matrix: which mistake does the model make?
3. Which mistake is more expensive? Choose recall, precision or F1 accordingly.
4. Pick the threshold on purpose, not just the default 0.5.

5. Use cross-validation for a fair, steady score, and always evaluate on data the model hasn't seen.

⚠️ Common mistakes

- Trusting accuracy on imbalanced data. A model that always says "no" can look excellent.
- Mixing up the order: score functions want `(real answers, predictions)`, real first.
- Forgetting which class is "positive". Precision and recall are about class `1` unless you say otherwise.
- Passing lists of different lengths → `ValueError: inconsistent numbers of samples`.
- Giving `roc_auc_score` the 0/1 predictions instead of the probabilities.
- Judging a model from one lucky (or unlucky) test split. Use cross-validation.

✅ Recap

- Confusion matrix: `tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()`.
- Precision = TP / (TP + FP): how trustworthy the alarms are.
- Recall = TP / (TP + FN): how many real cases were found.
- F1 = 2·P·R / (P + R): high only if both are high. `classification_report` shows them all.
- The threshold (default 0.5) trades precision for recall: `(proba >= t).astype(int)`.
- `cross_val_score(model, X, y, cv=5)` gives 5 scores; report their mean.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Confusion counts

A made-up test for a disease was tried on 20 people (`1` = sick). Use `confusion_matrix` to fill in the four variables `tn`, `fp`, `fn` and `tp`.

```
from sklearn.metrics import confusion_matrix

y_true = [0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0]
y_pred = [0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0]

# Fill in tn, fp, fn, tp
```

Hints:

1. `confusion_matrix(y_true, y_pred)` gives the 2 × 2 table. Real answers come first.
2. `.ravel()` flattens it into 4 numbers in the order TN, FP, FN, TP.
3. `tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()`

2. Precision and recall by hand

A made-up spam filter was tested on 200 emails ("spam" is the positive class). Using the formulas from today and the four variables, calculate `precision` and `recall`. Then print them with 2 decimals, like `Precision: 0.84` and `Recall: 0.75`.

Your code should still work if someone changes the four numbers.

```
tp = 42    # spam correctly sent to the spam folder
fp = 8     # good emails wrongly sent to spam
fn = 14    # spam that reached the inbox
tn = 136   # good emails correctly kept

# Calculate precision and recall, then print them
```

Hints:

1. Precision looks at everything the filter *called* spam: `tp / (tp + fp)`.
2. Recall looks at all the *real* spam: `tp / (tp + fn)`. Don't forget the brackets.
3. `print(f"Precision: {precision:.2f}")`, and the same for recall.

3. F1, two ways

For the made-up predictions below, calculate F1 twice:

- `f1_by_hand`: use `precision_score` and `recall_score`, then the formula `2 × P × R / (P + R)`
- `f1`: use scikit-learn's `f1_score`

The two numbers should be the same.

```
from sklearn.metrics import precision_score, recall_score, f1_score

y_true = [1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0]
y_pred = [1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0]

# Calculate f1_by_hand and f1
```

Hints:

1. `p = precision_score(y_true, y_pred)` and `r = recall_score(y_true, y_pred)`
2. `f1_by_hand = 2 * p * r / (p + r)`. The brackets around `p + r` matter.
3. `f1 = f1_score(y_true, y_pred)`

4. Cross-validation score

Use `cross_val_score` with `cv=5` on the whole `X` and `y` to score the pipeline `model`. Store the 5 scores in `scores` and their mean in `mean_score`.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer(as_frame=True)
X = cancer.data
y = (cancer.target == 0).astype(int)      # 1 = malignant

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))

# Calculate scores and mean_score
```

Hints:

1. You don't need `train_test_split` or `fit`: cross-validation does both for you.
2. `scores = cross_val_score(model, X, y, cv=5)`
3. `mean_score = scores.mean()`

5. Challenge: a threshold that misses few cases

For a first screening, the clinic wants a **recall of at least 0.95** on the test set, but with as few false alarms as possible. That means: the **highest** threshold that still reaches recall ≥ 0.95.

Go through `thresholds` (they are sorted from high to low). For each one, turn `proba` into 0/1 predictions and calculate the recall. Stop at the first threshold with recall ≥ 0.95, and store it in `best_threshold` and its recall in `best_recall`.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import recall_score

cancer = load_breast_cancer(as_frame=True)
X = cancer.data
y = (cancer.target == 0).astype(int)      # 1 = malignant
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1]

thresholds = [0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1]

# Find best_threshold and best_recall
```

Hints:

1. Loop: `for t in thresholds:`. Inside, `pred_t = (proba >= t).astype(int)`.
2. `r = recall_score(y_test, pred_t)`. Then `if r >= 0.95:` store `t` and `r`.
3. Inside the `if`: `best_threshold = t`, `best_recall = r`, then `break` to stop the loop (Day 5).

➡️ Tomorrow

Clustering with K-means: every model so far learned from examples with the right answers attached. Tomorrow there are no answers at all: you'll let the computer find groups by itself, like customer segments, with K-means and the elbow method.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Confusion counts

```
from sklearn.metrics import confusion_matrix

y_true = [0, 0, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0]
y_pred = [0, 0, 1, 0, 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0]

tn, fp, fn, tp = confusion_matrix(y_true, y_pred).ravel()
print(f"TN={tn}  FP={fp}  FN={fn}  TP={tp}")
```

2. Precision and recall by hand

```
tp = 42    # spam correctly sent to the spam folder
fp = 8     # good emails wrongly sent to spam
fn = 14    # spam that reached the inbox
tn = 136   # good emails correctly kept

# Calculate precision and recall, then print them
precision = tp / (tp + fp)
recall = tp / (tp + fn)
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
```

3. F1, two ways

```
from sklearn.metrics import precision_score, recall_score, f1_score

y_true = [1, 0, 1, 1, 0, 0, 1, 0, 0, 1, 0, 1, 0, 0, 1, 0]
y_pred = [1, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0]

p = precision_score(y_true, y_pred)
r = recall_score(y_true, y_pred)
f1_by_hand = 2 * p * r / (p + r)
f1 = f1_score(y_true, y_pred)
print(f"By hand: {f1_by_hand:.3f}   sklearn: {f1:.3f}")
```

4. Cross-validation score

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

cancer = load_breast_cancer(as_frame=True)
X = cancer.data
y = (cancer.target == 0).astype(int)      # 1 = malignant

model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))

scores = cross_val_score(model, X, y, cv=5)
mean_score = scores.mean()
print("Scores:", scores.round(3))
print(f"Mean: {mean_score:.3f}")
```

5. Challenge: a threshold that misses few cases

```
import pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import recall_score

cancer = load_breast_cancer(as_frame=True)
X = cancer.data
y = (cancer.target == 0).astype(int)      # 1 = malignant
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1]

thresholds = [0.9, 0.8, 0.7, 0.6, 0.5, 0.4, 0.3, 0.2, 0.1]

for t in thresholds:
    pred_t = (proba >= t).astype(int)
    r = recall_score(y_test, pred_t)
    if r >= 0.95:
        best_threshold = t
        best_recall = r
        break

print(f"Best threshold: {best_threshold}  (recall {best_recall:.3f})")
```